LAPORAN AKHIR PRAKTIKUM

Mata Praktikum	**:** Rekayasa Perangkat Lunak 2
Kelas	**:** 4IA24
Praktikum ke-	**:** 2
Tanggal	**:** 23 Oktober 2026
Materi	**:** Object Oriented Programming (Java)
NPM	**:** 50423237
Nama	**:** Axl Regan Yohande
Ketua Asisten	**:** Ayu Fara
Nama Asisten	**:**
Paraf Asisten	**:**
Jumlah Lembar	**:** 8 Lembar



LABORATORIUM INFORMATIKA

UNIVERSITAS GUNADARMA

2026

Listing

```
package com.pengadaan.salesorder;

import com.pengadaan.salesorder.model.*;
import com.pengadaan.salesorder.service.DokumenService;
import java.util.ArrayList;
import java.util.List;

public class MainDemoOop {

    public static void main(String[] args) {
        System.out.println("================================================================================");
        System.out.println("   DEMO TUGAS OOP (JAVA 17) - SISTEM SALES ORDER PENGADAAN SEKOLAH");
        System.out.println("   Penyedia : " + DokumenService.NAMA_PERUSAHAAN);
        System.out.println("   Direktur : " + DokumenService.NAMA_DIREKTUR);

System.out.println("================================================================================\n");

        // ------------------------------------------------------------------
        // 1. DEMO ENCAPSULATION (ENKAPSULASI)
        // ------------------------------------------------------------------
        System.out.println(">>> 1. PEMBUKTIAN ENCAPSULATION (ENKAPSULASI & VALIDASI ATRIBUT):");
        System.out.println("   - Atribut class ItemBarang bersifat PRIVATE.");
        System.out.println("   - Nilai diatur melalui Setter dan diakses melalui Getter.\n");

        ItemBarang barang1 = new ItemBarang();
        barang1.setNamaBarang("Laptop Chromebook ASUS C214");
        barang1.setQty(15);
        barang1.setSatuan("Unit");
        barang1.setHargaSatuan(6500000);

        System.out.println("   [OK] Item 1 Dibuat via Setter:");
        System.out.println("   - Nama Barang : " + barang1.getNamaBarang());
        System.out.println("   - Jumlah (Qty): " + barang1.getQty() + " " + barang1.getSatuan());
        System.out.println("   - Harga Satuan: Rp " + String.format("%,.0f", barang1.getHargaSatuan()));
        System.out.println("   - Subtotal    : Rp " + String.format("%,.0f", barang1.getJumlah()));
        System.out.println();

        // ------------------------------------------------------------------
        // 2. DEMO INHERITANCE (PEWARISAN)
        // ------------------------------------------------------------------
        System.out.println(">>> 2. PEMBUKTIAN INHERITANCE (PEWARISAN CLASS):");
        System.out.println("   - Superclass Abstrak: Transaksi (kodeTransaksi, tanggal, keterangan)");
        System.out.println("   - Subclass 1: Pesanan extends Transaksi");
        System.out.println("   - Subclass 2: Pengeluaran extends Transaksi\n");

        Sekolah sekolah = new Sekolah();
        sekolah.setNamaSekolah("SMKN 1 Jakarta");
        sekolah.setAlamatSekolah("Jl. Budi Utomo No. 7, Jakarta Pusat");
        sekolah.setNamaPenerima("Drs. H. Mulyadi, M.Pd");
        sekolah.setTeleponSekolah("081234567890");

        ItemBarang barang2 = new ItemBarang();
        barang2.setNamaBarang("Proyektor Epson EB-X500");
        barang2.setQty(5);
        barang2.setSatuan("Unit");
        barang2.setHargaSatuan(5800000);

        List<ItemBarang> daftarItem = new ArrayList<>();
        daftarItem.add(barang1);
        daftarItem.add(barang2);

        // Subclass Pesanan mewarisi Transaksi
        Pesanan orderSekolah = new Pesanan();
        orderSekolah.setKodeTransaksi("SO-2026-001");
        orderSekolah.setNomorPesanan("SO-2026-001");
```

```
        orderSekolah.setTanggal("2026-05-10");
        orderSekolah.setTanggalPesanan("2026-05-10");
        orderSekolah.setKeterangan("Pengadaan Sarana Lab Komputer Sekolah Dana BOS");
        orderSekolah.setSekolah(sekolah);
        orderSekolah.setItems(daftarItem);
        orderSekolah.hitungTotalOtomatis();

        // Subclass Pengeluaran mewarisi Transaksi
        Pengeluaran biayaLogistik = new Pengeluaran("2026-05-11", "Ongkos Ekspedisi Truk ke SMKN 1 Jakarta",
1500000, "Logistik & Transport");

        System.out.println("   Subclass Pesanan mewarisi atribut Transaksi:");
        System.out.println("   - Kode Transaksi (dari Superclass Transaksi) : " +
orderSekolah.getKodeTransaksi());
        System.out.println("   - Tanggal Transaksi (dari Superclass)       : " + orderSekolah.getTanggal());
        System.out.println("   - Sekolah Penerima (Khusus Subclass Pesanan): " +
orderSekolah.getSekolah().getNamaSekolah());
        System.out.println("   - Subtotal Nilai Barang                     : Rp " + String.format("%,.0f",
orderSekolah.getSubtotal()));
        System.out.println();

        System.out.println("   Subclass Pengeluaran mewarisi atribut Transaksi:");
        System.out.println("   - Kode Transaksi (dari Superclass Transaksi) : " +
biayaLogistik.getKodeTransaksi());
        System.out.println("   - Keterangan (dari Superclass)              : " +
biayaLogistik.getKeterangan());
        System.out.println("   - Nominal Biaya                             : Rp " + String.format("%,.0f",
biayaLogistik.getNominal()));
        System.out.println();

        // ---------------------------------------------------------------------
        // 3. DEMO POLYMORPHISM (POLIMORFISME)
        // ---------------------------------------------------------------------
        System.out.println(">>> 3. PEMBUKTIAN POLYMORPHISM (POLIMORFISME):");

        // A. Polimorfisme Dinamis (Method Overriding / Runtime)
        System.out.println("   A. Polimorfisme Dinamis (Method Overriding):");
        System.out.println("      Array Transaksi[] dapat menampung Pesanan maupun Pengeluaran.");
        System.out.println("      Method getJenisTransaksi() dieksekusi dinamis sesuai objek aslinya:\n");

        Transaksi[] daftarTransaksi = new Transaksi[] { orderSekolah, biayaLogistik };

        for (int i = 0; i < daftarTransaksi.length; i++) {
            Transaksi t = daftarTransaksi[i];
            System.out.println("      Transaksi ke-" + (i + 1) + " (Tipe Objek: " +
t.getClass().getSimpleName() + "):");
            System.out.println("      * Jenis Transaksi  : " + t.getJenisTransaksi());
            System.out.println("      * Deskripsi Ringkas: " + t.getDeskripsiRingkas());
            System.out.println("      * Total Nilai      : Rp " + String.format("%,.0f", t.getTotalNilai()));
            System.out.println("      ----------------------------------------------------------------");
        }

        // B. Polimorfisme Statis (Method Overloading / Compile-time)
        System.out.println("\n   B. Polimorfisme Statis (Method Overloading):");
        System.out.println("      Method hitungTotalOtomatis() memiliki 2 signature:");

        orderSekolah.hitungTotalOtomatis();
        System.out.println("      1. Tanpa parameter -> hitungTotalOtomatis() [Default Pajak PPN 11%]:");
        System.out.println("         Total Bayar = Rp " + String.format("%,.0f", orderSekolah.getTotal()));

        orderSekolah.hitungTotalOtomatis(12.0, 2.0);
        System.out.println("      2. Dengan parameter -> hitungTotalOtomatis(12.0, 2.0) [Overloading Pajak
Kustom]:");
        System.out.println("         Total Bayar = Rp " + String.format("%,.0f", orderSekolah.getTotal()));


System.out.println("\n================================================================================");
        System.out.println("   SUKSES: SEMUA BERJALAN DENGAN BAIK!");
```

```
        System.out.println("================================================================================");
    }
}
```

Logika

Pada praktikum ini saya membuat program demo OOP dengan Java 17 untuk sistem Sales Order pengadaan sekolah, yaitu class **MainDemoOop**. Program ini dipakai untuk membuktikan tiga prinsip OOP, yaitu **encapsulation**, **inheritance**, dan **polymorphism**. Semua pembuktian dijalankan dari satu method **main** supaya hasilnya bisa langsung terlihat berurutan di layar tanpa perlu membuka class lain.

Bagian paling atas adalah **package com.pengadaan.salesorder**, yang menandakan class ini berada di package utama aplikasi. Setelah itu ada beberapa import. Import **com.pengadaan.salesorder.model.*** saya pakai untuk mengambil seluruh class model sekaligus, yaitu **ItemBarang**, **Sekolah**, **Transaksi**, **Pesanan**, dan **Pengeluaran**. Import **DokumenService** diambil dari package service karena class ini menyimpan nama perusahaan dan nama direktur. Import **ArrayList** dan **List** dari java.util saya pakai untuk menyimpan daftar barang pada pesanan.

Method **main** dimulai dengan mencetak judul program dan garis pembatas dari tanda sama dengan. Setelah itu program mencetak nama penyedia dan nama direktur. Nilainya diambil dari konstanta **NAMA_PERUSAHAAN** dan **NAMA_DIREKTUR** milik **DokumenService**. Konstanta ini bisa dipanggil langsung lewat nama class tanpa membuat objek karena bersifat static.

Encapsulation: bagian pertama membuktikan bahwa atribut pada **ItemBarang** bersifat private, sehingga tidak bisa diubah atau dibaca langsung dari luar class. Satu-satunya jalan adalah lewat method setter untuk mengisi nilai dan method getter untuk membaca nilai. Cara ini membuat data lebih aman karena class bisa memeriksa nilai sebelum disimpan, sesuai dengan tulisan di judul bagian ini, yaitu validasi atribut.

Saya membuat objek **barang1** dengan constructor kosong, lalu mengisinya satu per satu. **setNamaBarang** diisi Laptop Chromebook ASUS C214, **setQty** diisi 15, **setSatuan** diisi Unit, dan **setHargaSatuan** diisi 6.500.000. Setelah itu nilainya dibaca kembali dengan **getNamaBarang**, **getQty**, **getSatuan**, dan **getHargaSatuan**. Untuk subtotal, program memanggil **getJumlah()** yang menghitung qty dikali harga satuan, yaitu 15 × 6.500.000 = 97.500.000. Format **%,.0f** pada **String.format** saya pakai supaya angka tampil dengan pemisah ribuan dan tanpa angka di belakang koma.

Inheritance: bagian kedua membuktikan pewarisan class. **Transaksi** adalah superclass abstrak yang menyimpan tiga atribut umum, yaitu **kodeTransaksi**, **tanggal**, dan **keterangan**. Class **Pesanan** dan **Pengeluaran** sama-sama **extends Transaksi**, sehingga keduanya otomatis memiliki ketiga atribut tersebut tanpa perlu menulis ulang. Keuntungannya, kode yang sama tidak ditulis dua kali, dan perubahan pada superclass langsung berlaku untuk kedua subclass.

Sebelum membuat pesanan, saya menyiapkan datanya terlebih dahulu. Objek **Sekolah** diisi dengan nama SMKN 1 Jakarta, alamat Jl. Budi Utomo No. 7 Jakarta Pusat, nama penerima Drs. H. Mulyadi, M.Pd, dan nomor telepon 081234567890. Barang kedua, yaitu **barang2**, adalah Proyektor Epson EB-X500 sebanyak 5 unit dengan harga satuan 5.800.000, sehingga subtotalnya 5 × 5.800.000 = 29.000.000. Kedua barang saya masukkan ke **List<ItemBarang> daftarItem** memakai **add**, sehingga satu pesanan bisa memuat lebih dari satu barang.

Objek **orderSekolah** bertipe **Pesanan** kemudian saya isi. **setKodeTransaksi**, **setTanggal**, dan **setKeterangan** adalah method warisan dari **Transaksi**. Sementara itu **setNomorPesanan**, **setTanggalPesanan**, **setSekolah**, dan **setItems** adalah method khusus milik **Pesanan**. Setelah semua data terisi, saya memanggil **hitungTotalOtomatis()**. Method ini menjumlahkan seluruh barang, yaitu 97.500.000 + 29.000.000 = 126.500.000, lalu menambahkan pajak untuk mendapatkan total bayar.

Objek **biayaLogistik** bertipe **Pengeluaran** dibuat dengan constructor yang punya empat parameter, yaitu tanggal 2026-05-11, keterangan Ongkos Ekspedisi Truk ke SMKN 1 Jakarta, nominal 1.500.000, dan kategori Logistik & Transport. Program lalu mencetak kode, tanggal, sekolah penerima, dan subtotal milik **orderSekolah**, serta kode, keterangan, dan nominal milik **biayaLogistik**. Dari hasil cetakan ini terlihat bahwa kode dan keterangan berasal dari superclass, sedangkan sekolah dan subtotal hanya ada di **Pesanan**, dan nominal hanya ada di **Pengeluaran**.

Polymorphism dinamis (overriding): bagian ketiga dimulai dengan membuat array **Transaksi[] daftarTransaksi** yang berisi **orderSekolah** dan **biayaLogistik**. Array ini bisa menampung keduanya karena **Pesanan** dan **Pengeluaran** sama-sama turunan **Transaksi**, sehingga satu variabel dengan tipe superclass dapat menunjuk ke objek subclass yang berbeda.

Selanjutnya perulangan **for** membaca setiap elemen array ke variabel **t** bertipe **Transaksi**. Pada tiap putaran, program memanggil **getJenisTransaksi()**, **getDeskripsiRingkas()**, dan **getTotalNilai()**. Walaupun variabelnya bertipe **Transaksi**, Java menjalankan versi method milik

objek aslinya saat program berjalan. Karena itu hasilnya berbeda untuk **Pesanan** dan **Pengeluaran**. Tipe asli objek ditampilkan dengan **getClass().getSimpleName()**. Total nilai pesanan adalah subtotal 126.500.000 ditambah PPN 11% sebesar 13.915.000, yaitu 140.415.000, sedangkan total nilai pengeluaran adalah 1.500.000.

Polymorphism statis (overloading): method **hitungTotalOtomatis()** punya dua signature dengan nama yang sama tetapi parameter yang berbeda. Versi tanpa parameter memakai pajak bawaan, yaitu PPN 11%, sehingga total bayar yang tercetak adalah 140.415.000. Versi dengan parameter **(12.0, 2.0)** memakai PPN 12% dan PPh 2%. PPN-nya 12% × 126.500.000 = 15.180.000 dan PPh-nya 2% × 126.500.000 = 2.530.000. Total bayarnya adalah subtotal ditambah PPN lalu dikurangi PPh, yaitu 126.500.000 + 15.180.000 − 2.530.000 = 139.150.000. Berbeda dengan overriding, pemilihan versi method di sini sudah ditentukan compiler dari jumlah parameter yang dikirim saat kompilasi.

Di akhir program, **main** mencetak garis pembatas dan pesan bahwa semua prinsip OOP telah terbukti. Jadi hasil keseluruhannya adalah sebagai berikut. Pada encapsulation, data **ItemBarang** diisi dan dibaca lewat setter dan getter dengan subtotal Rp 97.500.000. Pada inheritance, **Pesanan** dan **Pengeluaran** memakai atribut kode, tanggal, dan keterangan dari **Transaksi**, dengan subtotal pesanan Rp 126.500.000. Pada polymorphism dinamis, array **Transaksi[]** menampilkan hasil berbeda untuk **Pesanan** dan **Pengeluaran** sesuai objek aslinya. Pada polymorphism statis, **hitungTotalOtomatis()** menghasilkan total Rp 140.415.000 dengan PPN 11% dan Rp 139.150.000 dengan PPN 12% serta PPh 2%.

Output

```
================================================================================
   DEMO TUGAS OOP (JAVA 17) - SISTEM SALES ORDER PENGADAAN SEKOLAH
   Penyedia : CV Andalan
   Direktur : Axl
================================================================================

>>> 1. PEMBUKTIAN ENCAPSULATION (ENKAPSULASI & VALIDASI ATRIBUT):
   - Atribut class ItemBarang bersifat PRIVATE.
   - Nilai diatur melalui Setter dan diakses melalui Getter.

   [OK] Item 1 Dibuat via Setter:
   - Nama Barang : Laptop Chromebook ASUS C214
   - Jumlah (Qty): 15 Unit
   - Harga Satuan: Rp 6.500.000
   - Subtotal    : Rp 97.500.000

>>> 2. PEMBUKTIAN INHERITANCE (PEWARISAN CLASS):
   - Superclass Abstrak: Transaksi (kodeTransaksi, tanggal, keterangan)
   - Subclass 1: Pesanan extends Transaksi
   - Subclass 2: Pengeluaran extends Transaksi

   Subclass Pesanan mewarisi atribut Transaksi:
   - Kode Transaksi (dari Superclass Transaksi) : SO-2026-001
   - Tanggal Transaksi (dari Superclass)       : 2026-05-10
   - Sekolah Penerima (Khusus Subclass Pesanan): SMKN 1 Jakarta
   - Subtotal Nilai Barang                     : Rp 126.500.000

   Subclass Pengeluaran mewarisi atribut Transaksi:
   - Kode Transaksi (dari Superclass Transaksi) : EXP-2026-001
   - Keterangan (dari Superclass)               : Ongkos Ekspedisi Truk ke SMKN 1 Jakarta
   - Nominal Biaya                              : Rp 1.500.000

>>> 3. PEMBUKTIAN POLYMORPHISM (POLIMORFISME):
   A. Polimorfisme Dinamis (Method Overriding):
      Array Transaksi[] dapat menampung Pesanan maupun Pengeluaran.
      Method getJenisTransaksi() dieksekusi dinamis sesuai objek aslinya:

      Transaksi ke-1 (Tipe Objek: Pesanan):
      * Jenis Transaksi  : Pesanan Penjualan
      * Deskripsi Ringkas: Pesanan SO-2026-001 - SMKN 1 Jakarta
      * Total Nilai      : Rp 140.415.000
      ----------------------------------------------------------------
      Transaksi ke-2 (Tipe Objek: Pengeluaran):
      * Jenis Transaksi  : Pengeluaran Operasional
      * Deskripsi Ringkas: Logistik & Transport - Ongkos Ekspedisi Truk ke SMKN 1 Jakarta
      * Total Nilai      : Rp 1.500.000
      ----------------------------------------------------------------

   B. Polimorfisme Statis (Method Overloading):
      Method hitungTotalOtomatis() memiliki 2 signature:
      1. Tanpa parameter -> hitungTotalOtomatis() [Default Pajak PPN 11%]:
         Total Bayar = Rp 140.415.000
      2. Dengan parameter -> hitungTotalOtomatis(12.0, 2.0) [Overloading Pajak Kustom]:
         Total Bayar = Rp 139.150.000

================================================================================
   SUKSES: SEMUA BERJALAN DENGAN BAIK!
================================================================================
```